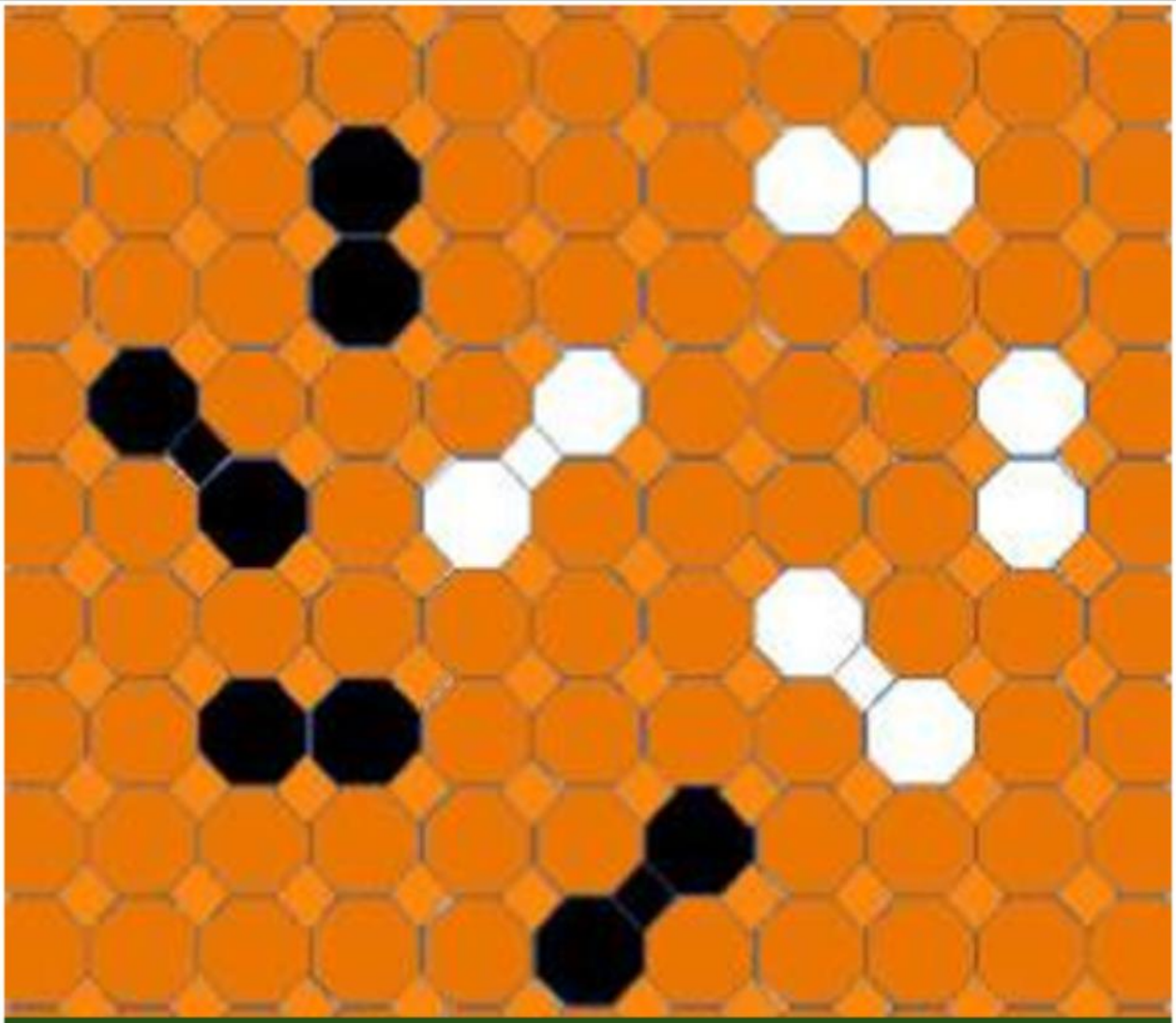


Quax Boardgame Feature Roadmap

Project Plan

Anna Magdalina
Kosei Masumi
Benjamin Ejike

Software Engineering 2

COMP20050

Group: 44 | Quax Boardgame | Sprint 4
Updated Roadmap & MVC Diagram

Completed Roadmap

Sprint	Features
1	1. A feature that displays the 11x11 Quax board layout with octagonal tiles (SR1) 2. A feature that displays rhombic tile positions on the board. (SR1) 3. A feature that allows the human player to place stones in empty octagonal tiles. (SR2) 4. A feature that displays whose turn is it (black/white) (eg. text "To Make a Move). Specifying if it is the Player's or Bot's turn also. (SR3)
2	1. A feature that enables the pie rule after Black's (bots) first move. (SR4) 2. A feature that allows White (player) to swap colours using the pie rule. (SR4) 3. A feature that allows placement of rhombic tiles between diagonally adjacent octagonal tiles with proper validation. (SR5) 4. A feature that detects and stops illegal moves on occupied tiles and displays an error message to the player. (SR6)
3	1. A feature that supports human vs bot gameplay by integrating a basic bot that makes a valid automatic move after the human players turn. (SR9) 2. A feature that detects orthogonal connections between stones of the same colour. (SR7) 3. A feature that detects diagonal connections using matching colour rhombic tiles. (SR7) 4. A feature that checks for Black top to bottom connections and displays a message indicating if Black won. (SR8) 5. A feature that checks for White left to right connections and displays a message indicating if White won. (SR8) 6. A feature that updates the UI to clearly display the bot's move after the human player's turn to distinguish bot actions from player actions. (SR9) 7. A feature that displays a clear "Game Over" message when the Player or the Bot wins. (SR10) 8. A feature that prevents further moves once a win condition has been reached. (SR10)
4	1. A feature that improves the bot behaviour to select better legal moves based on the current board state. (SR9)

We have implemented a better bot logic in this sprint:

- *A feature that improves the bot behaviour to select better legal moves based on the current board state. (SR9)*

The bot now makes more informed decisions rather than acting as a simple placeholder. It prioritises valid moves, attempts to advance its own path, and reacts more effectively to the current board state, making gameplay more realistic.

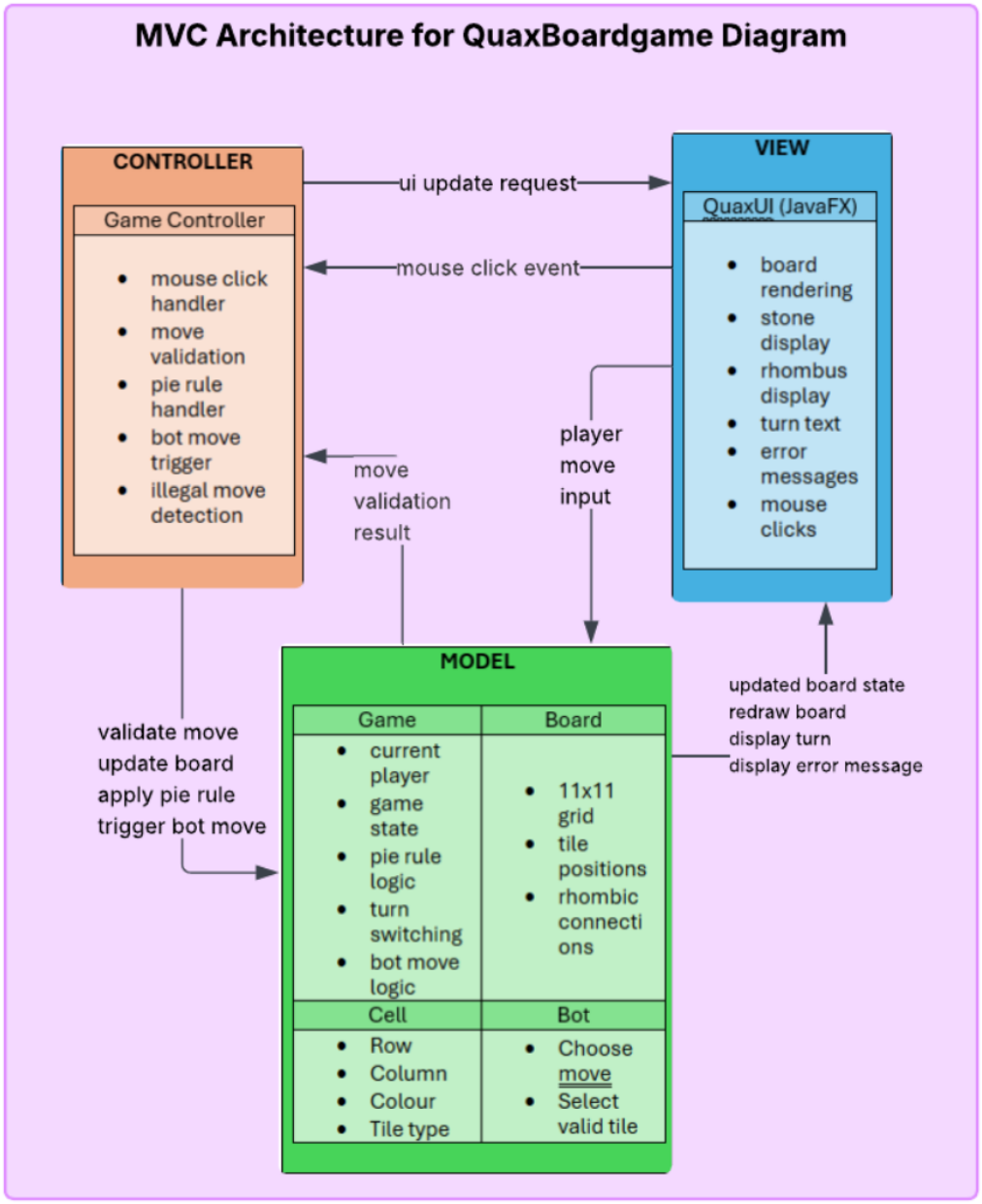
In addition to this, a small user interface enhancement was added:
Displays an opening click-to-play page with basic rules and winning conditions before the game begins. (UI enhancement – not part of original SRs).
This improves usability by giving the player a clearer starting point and explaining the objective of the game before interaction begins. It does not change gameplay logic and was added purely to improve the user experience.

MVC Architecture (For Quax Updated)

- The Model (Game + Board) handles all rules and state
- The View (QuaxUI) is only responsible for rendering and input
- The Controller behaviour is handled through method calls between UI and Game
- Key Methods moved / centralised into Game Class

The following logic is centralised into the Game class:

- placeStone(int row, int col): Previously, move validation and placement logic were partly handled in the UI. This is now fully handled in Game, including:
- Validating moves
- Placing stones
- Checking win conditions
- Switching turns
- placeRhombus(int row, int col): Rhombic placement validation and logic is now handled in Game, rather than directly in UI click handling.
- blackWins() and whiteWins(): Win detection logic (DFS search for connected paths) is now entirely in the Game class.
- hasMatchingRhombusBetween(...): Diagonal connection validation using rhombic tiles is now handled within game logic rather than UI.
- makeBotMove(): The bot's decision-making logic is now inside the Game class, instead of being controlled by the UI.


MVC Architecture for QuaxBoardgame Diagram
CONTROLLER
Game Controller
mouse click handler
move validation
pie rule handler
bot move trigger
illegal move detection
ui update request
mouse click event
VIEW
QuaxUI (JavaFX)
board rendering
stone display
rhombus display
turn text
error messages
mouse clicks
player move input
move validation result
MODEL
Game
current player
game state
pie rule logic
turn switching
bot move logic
Board
11x11 grid
tile positions
rhombic connections
Cell
Row
Column
Colour
Tile type
Bot
Choose move
Select valid tile
validate move
update board
apply pie rule
trigger bot move
updated board state
redraw board
display turn
display error message